
Confidence-Triggered Lifelong Adaptation for Drone Swarms Under Post-Training Surprises

Tanush Vijayakumar Savadi
UMass Amherst
tsavadi@umass.edu

Deveshi Singh
UMass Amherst
deveshisingh@umass.edu

Ron Kleinhouse-Goldman
UMass Amherst
rkleinhouseg@umass.edu

Code and artifacts: <https://github.com/tanushsavadi/confidence-triggered-swarm>

Abstract

Reinforcement learning policies for drone swarms are often trained in clean simulation, but deployment conditions can change through wind, sensor corruption, actuator degradation, and shifted goals. This project studies a two-drone formation task in PyBullet where a shared-policy Independent PPO controller is trained only on clean flight and then evaluated under post-training surprises. I build a confidence-triggered lifelong adaptation layer: entropy and Monte Carlo dropout variance are calibrated on clean rollouts, low-confidence usable episodes are used for between-episode reward-weighted updates, and KL anchoring, clean replay, and Elastic Weight Consolidation protect the original policy. The final validation evaluates five methods over three controlled evaluation seeds and 75 episodes per severity. Using paired seed-level percent changes, the best clean-retaining tuned variant, reward-weighted rescue, nearly preserves clean reward (-0.33%) and improves reward under mild ($+17.8\%$) and severe ($+39.2\%$) surprise, but it substantially underperforms the frozen policy under moderate surprise (-31.5%). Success rates remain near zero. The result is therefore mixed but informative: confidence-triggered adaptation gives partial reward recovery without clear clean collapse in this simplified setting, while the moderate condition exposes a real stability-plasticity failure.

1 Introduction

Drone swarms are useful because multiple agents can distribute sensing and control across space, but learned swarm policies are still brittle when the deployment setting differs from the simulator used for training. A controller that works in a clean formation-flight task may face wind, noisy observations, reduced actuator authority, or a changed waypoint after training. Retraining from scratch is expensive, while unrestricted online updates can overwrite the behavior that made the original policy useful.

This project asks a narrower question: can a clean-trained drone-swarm policy adapt to post-training surprises while keeping its clean-task skill? I train a shared-parameter Independent PPO (IPPO) policy for two Crazyflie-style drones in a formation waypoint task. The baseline is not trained with surprise randomization. After training, I inject controlled perturbations and compare a frozen policy against conservative confidence-triggered adaptation, a no-gate always-adapt baseline, and tuned adaptation variants.

The novelty is not any one component by itself. Entropy, dropout uncertainty, replay, KL regularization, and EWC all have prior roots. The contribution is the integrated pipeline and evaluation framing: a clean-trained formation policy, a post-training surprise suite, a dual-signal adaptation trigger, reward-weighted between-episode updates, explicit anti-forgetting safeguards, and a sequential continual-learning matrix that re-tests clean performance after later surprise phases. That last piece

matters because reward recovery under surprise would be less useful if the original clean behavior disappeared.

Concretely, I make four contributions:

- a post-training surprise suite for two-drone formation flight, with wind, sensor noise/dropout, actuator weakening, and waypoint shifts added only after clean training;
- a confidence-triggered adaptation loop that updates between episodes from usable low-confidence experience while anchoring to the clean policy;
- an extension evaluation protocol that runs frozen, current lifelong, always-adapt, improved PPO adaptation, and reward-weighted rescue across three controlled evaluation seeds;
- supporting diagnostics for forgetting, ablations, and a sequential continual-learning matrix.

2 Related work

Continual learning and forgetting. Continual learning studies how a model can learn from changing data without catastrophically forgetting previous tasks. The survey-style chapter by van de Ven et al. [2024], which I used as the main continual-learning framing reference, describes continual learning as learning from a non-stationary stream while retaining prior knowledge. That framing fits this project more closely than ordinary robustness evaluation: the question is not only whether surprise reward improves, but whether clean behavior survives after later adaptation.

Regularization and replay are standard ways to reduce forgetting. Elastic Weight Consolidation (EWC) penalizes changes to parameters estimated to be important for older tasks [Kirkpatrick et al., 2017]. Replay-based approaches mix earlier experience into later updates [Rolnick et al., 2019], and policy consolidation adapts related ideas to reinforcement learning [Kaplanis et al., 2019]. I use these ideas pragmatically: EWC, clean replay, and KL anchoring are not presented as new algorithms, but as safeguards around the post-training adaptation update.

Multi-agent PPO and drone simulation. PPO is the base policy-gradient method used here [Schulman et al., 2017]. In multi-agent reinforcement learning, PPO-style methods are strong practical baselines: Yu et al. [2022] show that PPO-based multi-agent methods can be competitive in cooperative settings when implemented carefully. My controller is closer to IPPO than MAPPO: both drones share actor-critic parameters and act from local observations, but I do not use a centralized critic. The simulator builds on gym-pybullet-drones, which provides PyBullet-based quadrotor environments for reinforcement learning [Panerati et al., 2021].

Robust RL and sim-to-real. Domain randomization exposes policies to variation during training so that deployment shifts are less surprising [Tobin et al., 2017]. Other robust-RL work trains against model ensembles or adversaries to improve generalization under dynamics changes [Rajeswaran et al., 2017, Pinto et al., 2017]. This project is intentionally different. The baseline is clean-trained, and surprises are introduced after training. That makes the setup less powerful than domain randomization, but it directly tests whether post-hoc confidence-triggered adaptation can recover some performance without destroying the clean skill.

Uncertainty and confidence in RL. Uncertainty estimates have been used for exploration, safety, and out-of-distribution detection [Osband et al., 2016, Lutjens et al., 2019]. MC dropout can be interpreted as an approximate Bayesian uncertainty estimate for neural networks [Gal and Ghahramani, 2016]. Here, uncertainty is not the exploration objective. It is a trigger that decides whether recent deployment data should be used for adaptation.

3 Task and surprise suite

Formation task. The environment contains two CF2X drones in PyBullet. The swarm centroid must move through a short waypoint course while the drones maintain a target spacing of 0.5 m. Each episode lasts up to 15 simulated seconds, with physics at 240 Hz and control at 30 Hz. Each drone observes the base kinematic state from gym-pybullet-drones plus the relative offset to the current waypoint. The shared policy outputs velocity-level actions, and the simulator’s low-level controller converts those commands to drone control.

Table 1: Surprise presets used in the saved evaluation artifacts.

Severity	Wind max (N)	Sensor σ	Dropout	Actuator scale	Goal shift
clean	0.00	0.00	0.00	1.00	none
mild	0.02	0.01	0.00	1.00	none
moderate	0.05	0.02	0.02	0.85	none
severe	0.10	0.05	0.05	0.70	$p = 0.001, 0.1 \text{ m}$

The reward combines waypoint tracking, formation keeping, waypoint bonuses, collision penalties, a small survival reward, and penalties for unsafe altitude or excessive tilt. Episodes terminate when all waypoints are reached and truncate on timeout, out-of-bounds behavior, low altitude, or excessive tilt.

Surprise levels. The SurpriseWrapper injects wind forces, sensor noise, optional sensor dropout, actuator weakness, and stochastic waypoint shifts. These perturbations are used after baseline training, not during clean baseline training. Table 1 summarizes the canonical presets.

4 Method

Clean IPPO baseline. I use a shared-parameter IPPO controller. Both drones act from their own observations, but the actor-critic weights are shared. The policy is a two-layer MLP with 256 hidden units per layer, Tanh activations, a Gaussian actor head, a critic head, and dropout layers with probability 0.1. PPO uses clipped policy updates, generalized advantage estimation, entropy regularization, value loss, and gradient clipping [Schulman et al., 2017]. The canonical checkpoint is trained for one million timesteps with seed 42.

Confidence trigger. Before deployment, the confidence monitor is calibrated on clean episodes. At evaluation time it computes two signals for each observation: policy entropy and MC-dropout action variance from 10 stochastic forward passes. Each signal is normalized against the clean calibration statistics, mapped into a confidence value, and combined with a 70/30 weighting toward entropy. The rolling confidence window has length 30. Adaptation can fire when rolling confidence is low, or when the episode-level confidence is low and enough usable replay data exists.

Episode quality gate. Not every low-confidence episode should be learned from. Short crashes can teach the policy to imitate failure. An episode is added to the adaptation replay buffer only if it lasts at least 30 steps and has mean per-step reward at least -5.0 . This gate is conservative, but it keeps the adaptation loop from treating unusable failures as training data.

Reward-weighted adaptation. Adaptation happens between episodes. The main method uses reward-weighted behavior cloning over recent accepted transitions. Rewards are standardized, clipped, passed through a softmax weighting, and used to scale the log-probability objective so higher-rewarded transitions receive larger gradient weight. Each triggered update runs for five epochs with learning rate 10^{-4} .

Anti-forgetting safeguards. Three mechanisms protect the clean policy. First, a KL penalty keeps the current policy close to a frozen clean-policy copy. Second, 20% clean replay mixes calibration transitions into adaptation minibatches. Third, EWC penalizes movement away from clean-task parameters with high empirical Fisher information:

$$L_{EWC} = \frac{\lambda}{2} \sum_i F_i (\theta_i - \theta_i^*)^2, \quad \lambda = 1000. \quad (1)$$

5 Experiments

5.1 Experimental setup and reproducibility

The final extension validation uses three controlled evaluation seeds, 75 episodes per severity, and the same clean-trained checkpoint for all methods. The validation compares five methods: FROZEN, CURRENT LIFELONG, ALWAYS-ADAPT, IMPROVED PPO, and REWARD-WEIGHTED RESCUE. PyBullet resets, environment resets, surprise RNGs, NumPy, Python, and PyTorch are seeded through the evaluation path. The validation summary is generated from

Table 2: Final validation over three controlled evaluation seeds and 75 episodes per severity. Entries are reward mean $\pm$ standard error. All methods use the same clean-trained checkpoint.

Severity	Frozen	Current	Always-adapt	Improved PPO	Reward rescue
clean	2011.6 $\pm$ 0.0	1972.4 $\pm$ 10.9	1971.9 $\pm$ 3.2	1988.0 $\pm$ 19.9	2004.9 $\pm$ 23.6
mild	199.8 $\pm$ 16.9	235.7 $\pm$ 24.8	238.2 $\pm$ 26.4	241.5 $\pm$ 34.5	233.1 $\pm$ 34.7
moderate	119.3 $\pm$ 13.3	77.4 $\pm$ 10.6	77.3 $\pm$ 9.7	78.3 $\pm$ 11.0	77.7 $\pm$ 9.9
severe	30.5 $\pm$ 5.6	38.6 $\pm$ 3.9	38.1 $\pm$ 3.8	38.2 $\pm$ 4.0	38.3 $\pm$ 4.0

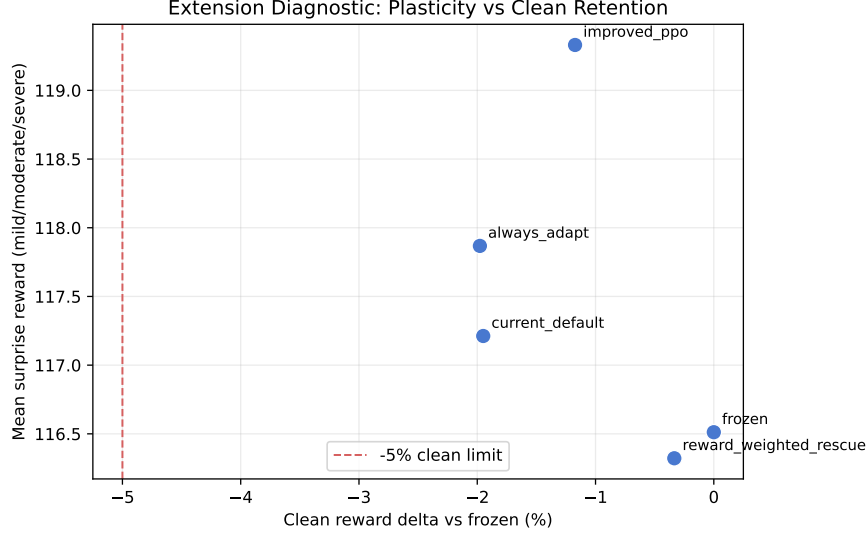


Figure 1: Reward recovery versus clean retention in the final validation. Adaptive methods cluster tightly: they improve mild and severe reward but do not fix the moderate condition.

runs/extension/validation/aggregate_summary.json; the corresponding figure is saved as runs/extension/validation/diagnostic_reward_retention.pdf. The clean baseline checkpoint is runs/baseline/best_model.pt.

The screening stage used 25 episodes per severity for candidate selection and is saved in runs/extension/screening/. The final validation then increased the episode count and used the same protocol for every retained method. This matters because deep-RL comparisons are sensitive to randomness, reporting choices, and implementation details [Henderson et al., 2018, Patterson et al., 2024]. I report mean reward with standard error over the three evaluation seeds. These are still not confidence intervals over independently trained policies: all final validation results reuse one trained checkpoint. That one-training-seed constraint is the main statistical limitation of the project.

5.2 Final extension validation

The clean-trained frozen policy degrades sharply under surprise. In the final validation, frozen reward falls from 2011.6 on clean to 199.8 on mild, 119.3 on moderate, and 30.5 on severe. The adaptive methods do not uniformly solve this degradation. They improve mild and severe reward, but all adaptive variants underperform frozen on moderate. Table 2 reports the full method comparison.

Using paired seed-level percent changes, the best clean-retaining tuned variant is REWARD-WEIGHTED RESCUE: clean reward changes by only -0.33% , mild improves by $+17.8\%$, and severe improves by $+39.2\%$ relative to frozen. IMPROVED PPO gives the best mild reward ($+22.1\%$) and similar severe recovery ($+39.0\%$), but has a larger clean drop (-1.17%). The difference between the tuned methods is small enough that the paper should not claim a decisive algorithmic win.

Two negative results are central. First, every adaptive method is worse than frozen on moderate surprise, with drops of roughly 31%. Second, success rates remain near zero across the final validation.

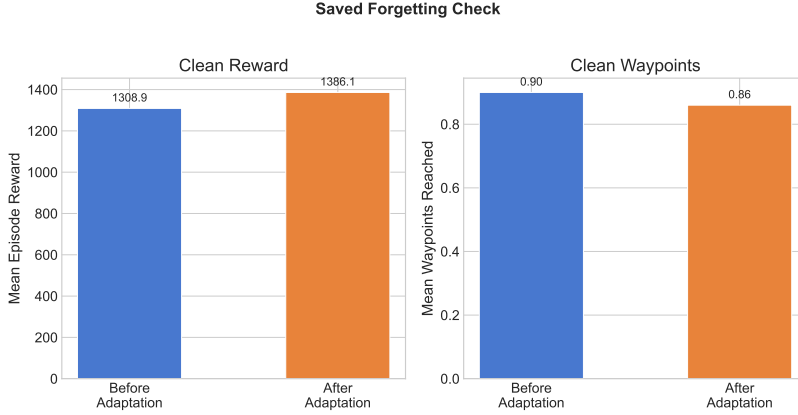


Figure 2: Clean reward before and after severe adaptation. The run does not show catastrophic forgetting, although the result is still single-seed evidence.

The method improves partial behavior, especially reward and waypoint progress, but it does not make the surprised task reliably successful.

ALWAYS-ADAPT is also informative. It adapts on every clean episode and much more often than the confidence-gated methods, yet it is not clearly better. This weakens any claim that more adaptation is sufficient, and supports the narrower interpretation that the safeguards and gates mainly control the stability-plasticity tradeoff rather than producing a dominant recovery method.

5.3 Single-seed supporting diagnostics

The final validation is the primary result. The remaining diagnostics were run earlier on the seed-42 checkpoint and should be read as supporting evidence about failure modes and forgetting, not as independent proof of generalization.

5.3.1 Forgetting probe

A useful adaptation method should not recover from surprise by erasing the original task. The saved forgetting probe adapts under severe surprise and then evaluates on clean. Clean reward is 1308.9 (std. 325.5) before the severe-adaptation phase and 1386.1 (std. 306.2) afterward. Mean waypoints reached moves from 0.90 to 0.86, while success rate moves from 0.02 to 0.04. The evaluator reports `forgetting_detected: false`.

The reward increase after adaptation should not be over-interpreted as proof that severe adaptation improves clean flight. The safer interpretation is that clean behavior does not collapse in this run. That is the key requirement for calling the system lifelong rather than only adaptive.

5.3.2 Sequential continual-learning matrix

The per-severity evaluator resets from the clean checkpoint for each condition. To test continual learning more directly, we also run one sequence: clean, then mild, then moderate, then severe. After each phase, the same lifelong policy is evaluated on every severity. This produces a matrix $R[i, j]$, where row i is the adaptation phase completed and column j is the evaluation condition. Matrix-style diagnostics support backward transfer, forward transfer, and remembering metrics used in continual-learning analyses [Lopez-Paz and Ranzato, 2017, van de Ven et al., 2024].

The clean column remains in the same range across phases: 1316.0, 1261.3, 1406.2, and 1322.9. The final average reward is 393.1 for lifelong and 412.9 for the frozen reference. Backward transfer is +15.4, forward transfer is +17.2, and remembering is 1.0. These numbers support clean retention and positive transfer metrics in this seed, but they do not show that lifelong adaptation dominates the frozen policy on every aggregate measure.

Table 3: Sequential continual-learning matrix for the lifelong policy. Rows are completed adaptation phases; columns are evaluation conditions.

Phase completed	Clean	Mild	Moderate	Severe
after clean	1316.0	146.9	48.1	38.5
after mild	1261.3	143.5	94.9	21.5
after moderate	1406.2	159.2	40.3	31.8
after severe	1322.9	131.2	92.0	26.1

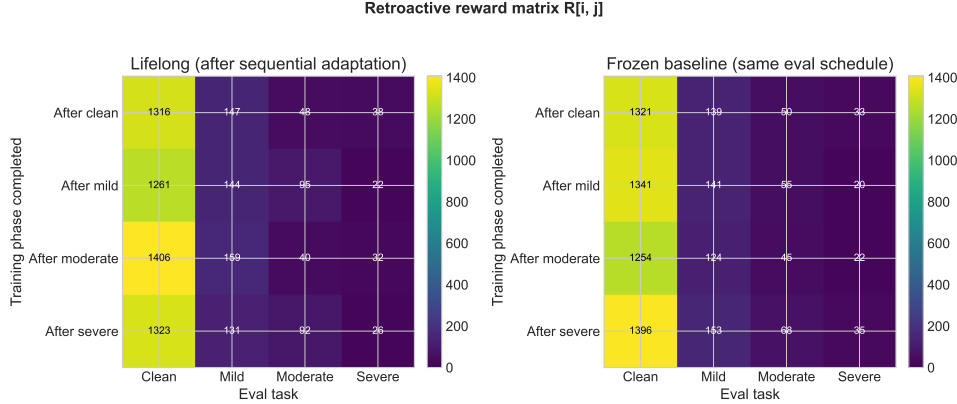


Figure 3: Retroactive reward matrices for lifelong and frozen policies. The clean column is re-evaluated after each later phase, which directly tests whether clean performance was retained.

5.3.3 Ablations

The ablation run evaluates severe surprise with 15 episodes per variant. Removing safeguards can increase mean reward, but the variance rises substantially. The full method has the lowest standard deviation in the ablation, which matches the design goal of conservative adaptation.

This ablation does not prove that every safeguard is necessary in every setting. It does show the tradeoff I cared about: the safeguards make the method cautious, and caution can reduce peak recovery. The full method also triggered zero adaptations in this shorter 15-episode ablation, while several ablated variants triggered once. That makes the ablation mainly a stability and trigger-sensitivity diagnostic, not a final ranking of severe-surprise performance.

6 Discussion, limitations, and safety

The main lesson is that post-training surprise is damaging for a clean-trained formation policy. Even mild wind and sensor noise sharply reduce frozen reward. The final validation shows partial recovery in mild and severe conditions with little clean degradation for the best clean-retaining tuned variant. That is the strongest supported claim.

The limits are just as important. Moderate surprise remains a failure case: all adaptive methods lose roughly one third of frozen reward in that condition. Success rates stay near zero, so the method improves shaped reward and waypoint progress rather than reliably completing the task. All final validation results also use one trained checkpoint, so the three seeds measure controlled evaluation variation, not variation across independently trained policies. The system is tested with two drones, not a large swarm. Severe absolute reward remains low even after adaptation. The confidence-gated variants adapt rarely in the final validation, which means the method is more conservative than responsive. The simulation is simplified, and adaptation happens only between episodes. The final continual average reward is also lower than the frozen reference, so the system is not a universal performance win.

There is also a safety limitation. This is simulation-only work and should not be read as a deployable drone-control method. In real hardware, online or post-episode adaptation could make unsafe behavior

Table 4: Severe-surprise ablation results. Higher means in ablated variants come with much higher variance.

Variant	Mean reward	Std. reward	Waypoints	Adaptations
Frozen	29.18	62.04	0.00	0
Full method	24.30	21.58	0.13	0
No KL anchor	53.13	77.84	0.07	1
No clean replay	46.21	59.91	0.27	1
No EWC	45.89	62.81	0.27	1

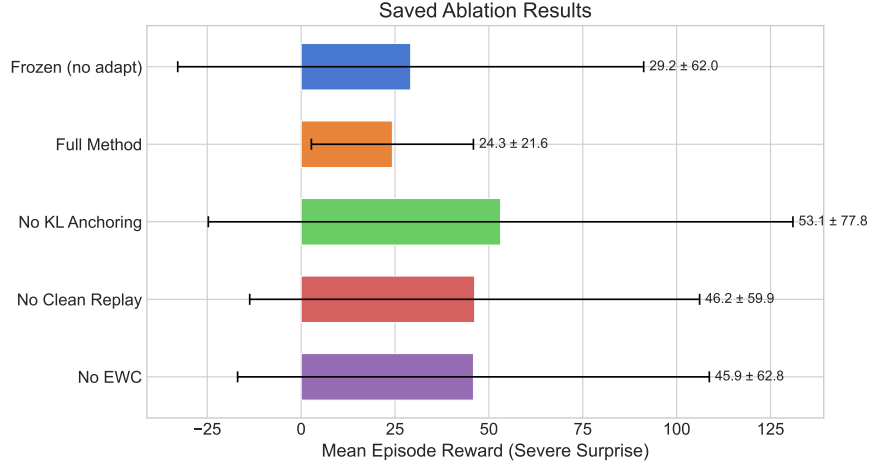


Figure 4: Ablation results under severe surprise. The full method trades peak reward for lower variance, while removing safeguards can produce higher but less stable reward.

worse if the confidence trigger, quality gate, or reward signal is wrong. A real deployment would need hard safety constraints, hardware limits, human oversight, and tests under many more environment variations before any autonomous adaptation could be trusted.

7 Author contributions

I, Tanush Vijayakumar Savadi, did the technical implementation, experiments, analysis, figure generation, check-in video, final presentation deck, and final report integration. This includes the environment setup, surprise wrapper, PPO/IPPO training code, confidence monitor, between-episode adaptation loop, anti-forgetting safeguards, seeded extension runner, validation aggregation, ablations, and continual-learning evaluation. Ron Kleinhause-Goldman looked over the proposal, provided proposal-stage comments/edits, and presented the proposal. After the proposal stage, I did not receive further contributions from Ron to the implementation, experiments, analysis, final report, or final presentation. Deveshi Singh did not contribute substantive implementation, experiments, analysis, writing, or presentation work to the final project.

8 Conclusion

I built a confidence-triggered lifelong adaptation system for a clean-trained two-drone formation policy under post-training surprises. The system combines a controlled surprise suite, entropy and MC-dropout confidence monitoring, reward-weighted between-episode updates, and anti-forgetting safeguards. The final validation is mixed: adaptive methods improve mild and severe reward relative to frozen and can preserve clean reward, but they consistently fail on moderate surprise and do not produce reliable task success. The next steps are independent training seeds, a stronger domain-randomized baseline, larger swarms, better trigger calibration, stronger adaptation objectives, peer-help mechanisms between drones, and eventually sim-to-real testing.

References

- Yarin Gal and Zoubin Ghahramani. Dropout as a bayesian approximation: Representing model uncertainty in deep learning. In *International Conference on Machine Learning*, 2016.
- Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, pages 3207–3214, 2018.
- Christos Kaplanis, Murray Shanahan, and Claudia Clopath. Policy consolidation for continual reinforcement learning. In *International Conference on Machine Learning*, 2019.
- James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.
- David Lopez-Paz and Marc’Aurelio Ranzato. Gradient episodic memory for continual learning. In *Advances in Neural Information Processing Systems*, 2017.
- Bjorn Lutjens, Michael Everett, and Jonathan P. How. Safe reinforcement learning with model uncertainty estimates. In *IEEE International Conference on Robotics and Automation*, 2019.
- Ian Osband, Charles Blundell, Alexander Pritzel, and Benjamin Van Roy. Deep exploration via bootstrapped dqn. In *Advances in Neural Information Processing Systems*, 2016.
- Jacopo Panerati, Hehui Zheng, SiQi Zhou, James Xu, Amanda Prorok, and Angela P. Schoellig. Learning to fly: a gym environment with pybullet physics for reinforcement learning of multi-agent quadrotor control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2021.
- Andrew Patterson, Samuel Neumann, Martha White, and Adam White. Empirical design in reinforcement learning. *Journal of Machine Learning Research*, 25(318):1–63, 2024.
- Lerrel Pinto, James Davidson, Rahul Sukthankar, and Abhinav Gupta. Robust adversarial reinforcement learning. In *International Conference on Machine Learning*, pages 2817–2826, 2017.
- Aravind Rajeswaran, Sarvjeet Ghotra, Balaraman Ravindran, and Sergey Levine. EPOpt: Learning robust neural network policies using model ensembles. In *International Conference on Learning Representations*, 2017.
- David Rolnick, Arun Ahuja, Jonathan Schwarz, Timothy Lillicrap, and Gregory Wayne. Experience replay for continual learning. In *Advances in Neural Information Processing Systems*, 2019.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2017.
- Gido M. van de Ven, Nicholas Soures, and Dhireesha Kudithipudi. Continual learning and catastrophic forgetting. *arXiv preprint arXiv:2403.05175*, 2024.
- Chao Yu, Akash Velu, Eugene Vinitzky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of PPO in cooperative multi-agent games. In *Advances in Neural Information Processing Systems*, volume 35, pages 24611–24624, 2022.